

Assignment 3 Report

Course 203.3630 – Artificial Intelligence Lab.

All numbers in this report come from the generated result files under ``results/final_experiments/`` (final run with seeds 42, 43, 44 from ``configs/final_experiment_plan.json``). Nothing was filled in by hand without a matching CSV row; ``scripts/extract_report_numbers.py`` prints the same facts for checking.

1. Introduction

The assignment has two parts. Part A asks for six search algorithms (Simulated Annealing, Tabu Search, Ant Colony Optimization, a Genetic Algorithm with an Island Model, Adaptive Large Neighborhood Search, and Branch & Bound / Limited Discrepancy Search), first validated on the continuous Ackley function as a warm-up and then compared on six official CVRP benchmark instances, together with an explicit multi-stage heuristic. Part B asks for generative AI: evolving Rush Hour heuristic functions with both Genetic Programming (GP) and Gene Expression Programming (GEP), where every candidate heuristic is judged by actually running A* with it.

All stochastic runs use the fixed seeds 42, 43 and 44 from the final experiment plan, so every number below is reproducible.

After the first complete run, a controlled tuning pass (validated on all six instances before acceptance) improved SA, GA-Island and ALNS, and Part B was strengthened with a measured 14-puzzle hard Rush Hour benchmark. The final tuned rerun improved the best CVRP gaps on three instances with no regressions, and the Ackley results were left unchanged. All numbers in this report come from that final tuned rerun.

2. Implementation Overview

2.1 Project structure

The code is a plain Python package: ``src/common/`` (timing), ``src/ackley/`` (function + the six adaptations), ``src/cvrp/`` (model, parser, cost, validation, baseline) with ``src/cvrp/solvers/`` (the six algorithms), ``src/rushhour/`` (board, A*, safe evaluator, GP/GEP comparison), ``src/gp/`` and ``src/gep/`` (separate frameworks), and ``src/experiments/`` (runners, summaries, report assets). ``scripts/`` holds the CLI entry points and ``tests/`` the pytest suite (268 tests at the time of the final run).

2.2 Reproducibility and command-line interface

Every run takes its inputs from CLI arguments – instance paths, seeds, budgets and timeouts are never hardcoded. Each experiment row is written to CSV with its seed, budget, timeout, costs, feasibility, errors and timing.

The final settings live in ``configs/final_experiment_plan.json`` and the runner ``scripts/run_final_experiments.py`` is resumable (finished raw CSVs are skipped on a rerun).

2.3 Validation and safety checks

Every CVRP solution is validated: routes start and end at the depot, no customer is missing or duplicated, capacity is respected, and the number of used routes must not exceed the vehicle count. Rows report ``feasible`` and the exact validation errors – nothing is silently fixed. Rush Hour heuristic evaluation runs A* under a node cap, a per-puzzle time cap and a total time budget, and an exception inside a candidate heuristic is recorded as a failed puzzle instead of crashing the run.

3. Part A – Ackley Function

The Ackley function for d dimensions:

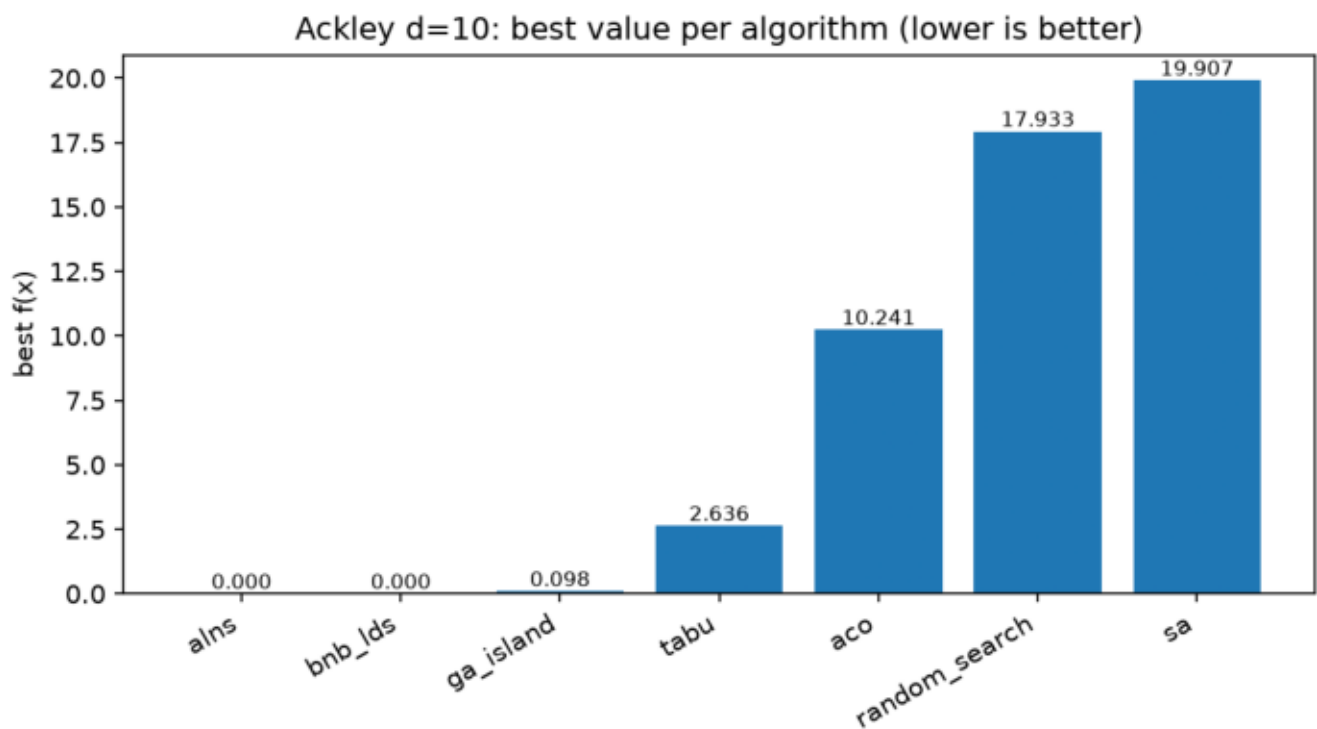
$$f(x) = -a * \exp(-b * \sqrt{(1/d) * \sum(x_i^2)}) - \exp((1/d) * \sum(\cos(c * x_i))) + a + e$$

with $a = 20$, $b = 0.2$, $c = 2\pi$, dimension $d = 10$, bounds x_i in $[-32.768, 32.768]$, and the known optimum $f(0, \dots, 0) = 0$.

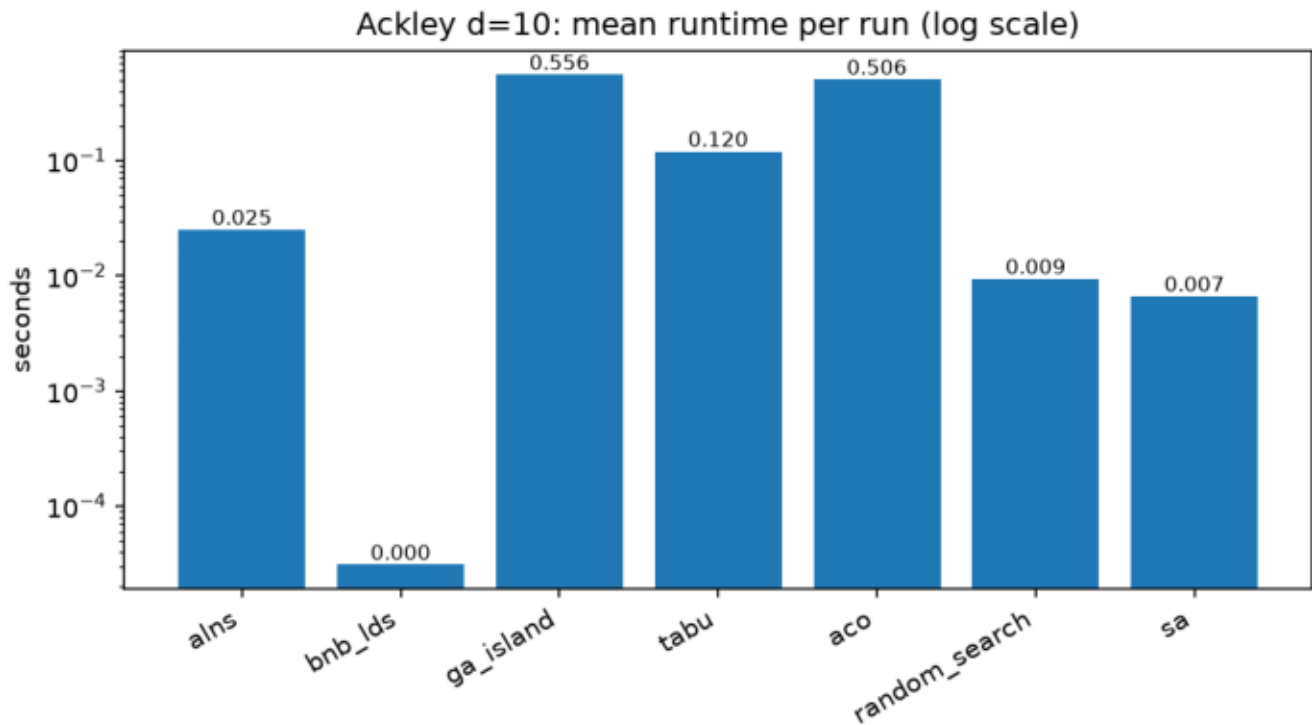
Final results (3 seeds per algorithm, budget 500, timeout 60 s), from

`summary/ackley\_d10\_summary.csv`:

algorithm	runs	best_value	mean_best_value	std_best_value	mean_dist_from_origin	mean_elapsed (s)
---	---	---	---	---	---	---
ackley_alns	3	0.000000	0.000000	0.000000	0.000000	0.025
ackley_bnb_lds	3	0.000000	0.000000	0.000000	0.000000	0.000
ackley_ga_island	3	0.098	0.100	0.002	0.063	0.556
ackley_tabu	3	2.636	3.018	0.359	1.466	0.120
ackley_aco	3	10.241	11.763	1.406	11.187	0.506
random_search	3	17.933	18.320	0.481	30.278	0.009
ackley_sa	3	19.907	20.041	0.118	46.706	0.007



The figure shows the best value each algorithm reached over its three seeds, sorted from best to worst. The spread is huge – from exactly 0 to almost 20 – which mostly reflects how well each adaptation fits a continuous function, not a universal ranking of the methods.



Runtime (log scale) shows the other side: SA and random search are almost free, while ACO and GA-Island pay for population/colony bookkeeping. Note that B&B/LDS looks instant only because its greedy zero-first bin order finds the origin immediately and the search stops early.

Short analysis. ALNS and B&B/LDS reached 0.000000 (six decimals) on every seed – but both benefit from knowing the optimum is at the origin: the ALNS "toward zero" repair operator and the B&B/LDS bin ranking by distance to zero point straight at it, so this says more about the adaptation than about general optimization strength. GA-Island got close (≈0.1) without such a hint. Tabu Search reached ≈2.6. ACO's coarse bins (10 per dimension) limit its precision. SA with the untuned parameters (initial temperature 10, step scale 1) actually ended worse than plain random search – an honest negative result: 500 local Gaussian steps from a random corner of a 10-dimensional box are simply not enough without parameter tuning.

4. Part A – CVRP

Official instances and best-known solutions (BKS):

instance	BKS cost
---	---
P-n16-k8	450
E-n22-k4	375
A-n32-k5	784
A-n80-k10	1763
X-n101-k25	27591
M-n200-k17	1275

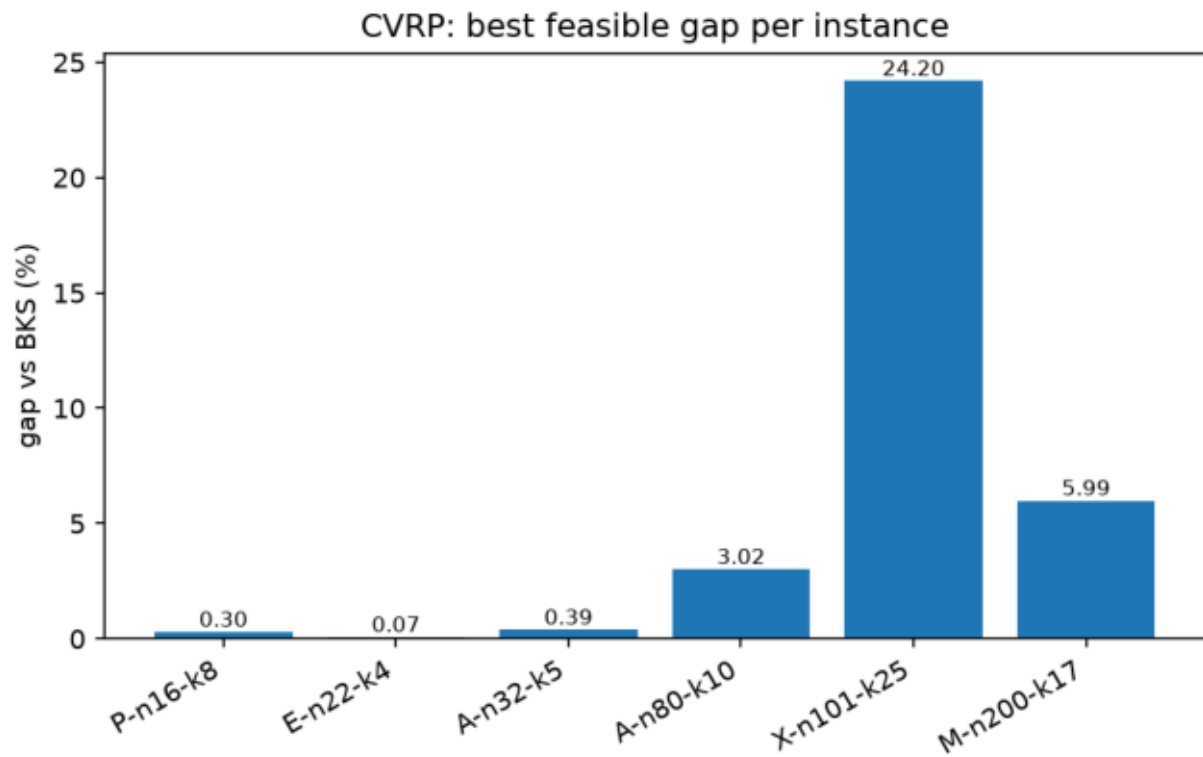
The BKS values are used only to compute the gap percentage $(100 \cdot (\text{cost} - \text{BKS}) / \text{BKS})$; the algorithms never see them.

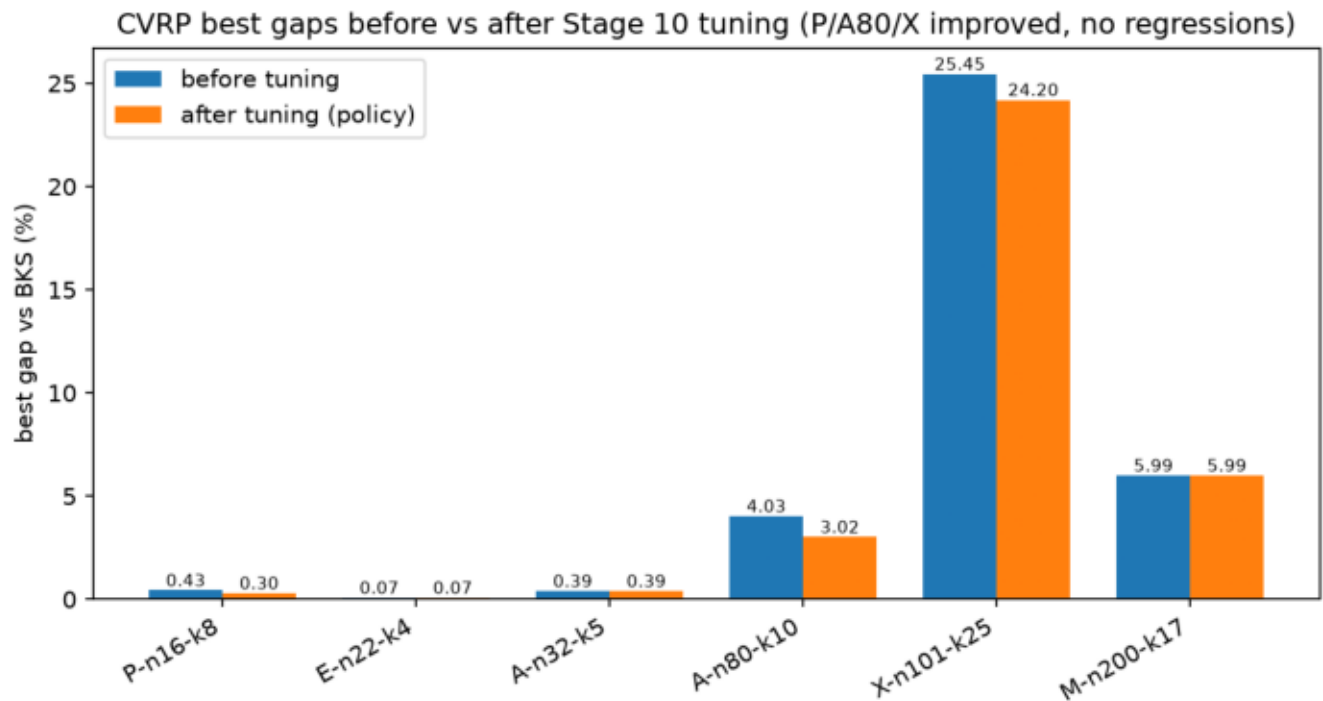
The final run uses tuned settings accepted after a separate validation pass: SA runs 50× more (very cheap) iterations with a slower cooling schedule and a higher start temperature; GA-Island uses population 30 with mutation 0.3; and ALNS gained extra destroy/repair operators (Shaw-style related removal, full-route removal, segment removal, regret-3 insertion). Because the enhanced ALNS regressed on M-n200-k17 during validation, the final rerun executes **both** ALNS variants (144 raw rows in total: 8 algorithm rows × 6 instances × 3 seeds) and the reported "alns" result

follows a rule declared **before** the rerun: enhanced everywhere except M-n200-k17, which uses the basic variant. Nothing is cherry-picked after the fact and both variants' raw rows are kept.

Best feasible result per instance (all 144 rows feasible):

instance	BKS	best algorithm	best cost	best gap	previous best
---	---	---	---	---	---
P-n16-k8	450	sa (tuned)	451.35	0.30%	0.43%
E-n22-k4	375	sa (tuned)	375.28	0.07%	0.07%
A-n32-k5	784	alns	787.08	0.39%	0.39%
A-n80-k10	1763	alns (enhanced)	1816.28	3.02%	4.03%
X-n101-k25	27591	ga_island (tuned)	34269.89	24.20%	25.45%
M-n200-k17	1275	alns	1351.43	5.99%	5.99%

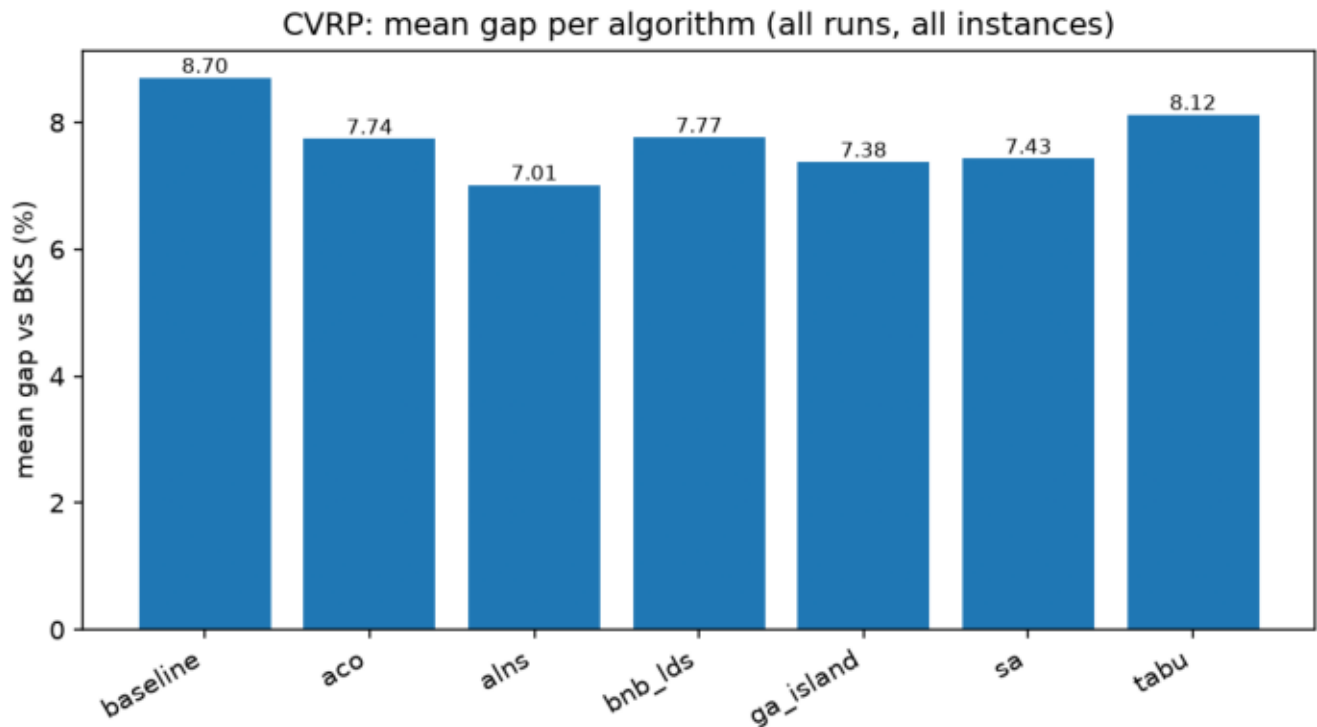




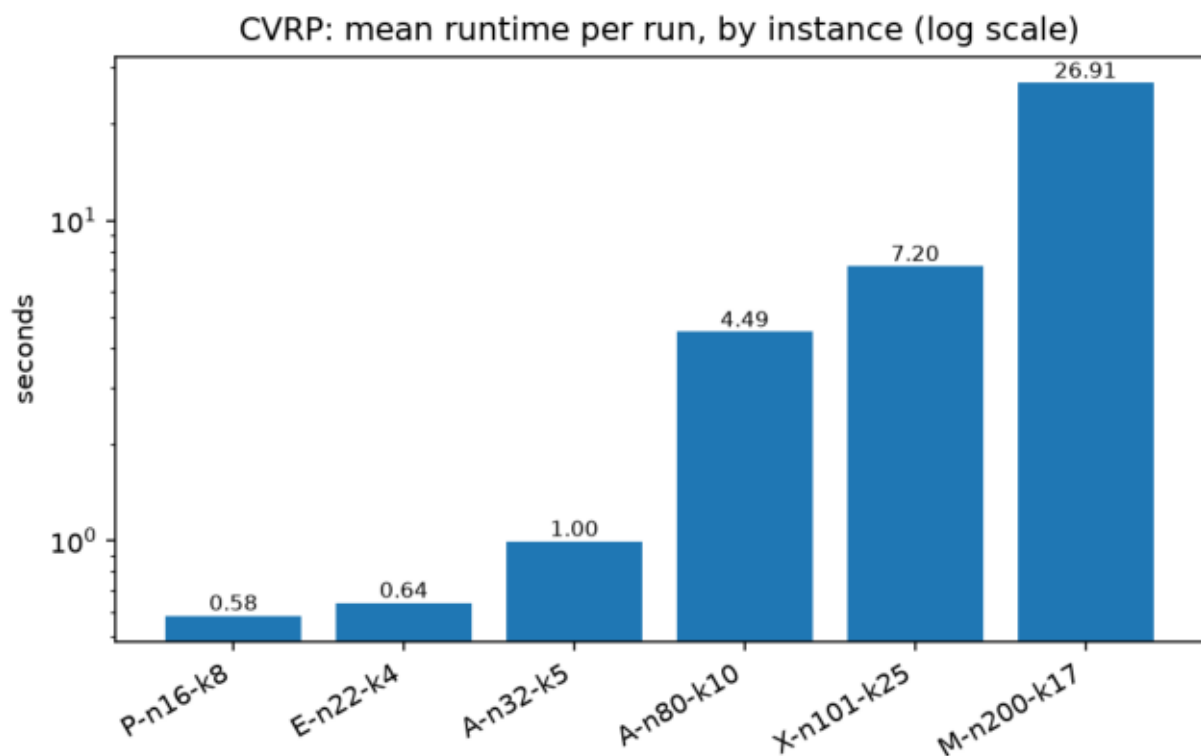
The before/after chart shows what the tuning pass actually bought: P-n16-k8 (0.43?0.30), A-n80-k10 (4.03?3.02) and X-n101-k25 (25.45?24.20) improved, the other three stayed exactly where they were, and nothing got worse – the pre-declared ALNS policy is what protected M-n200-k17. The X-n101-k25 column still stands out; that is a capacity-packing property of the instance, explained below, not an algorithm bug.

Per algorithm (mean of best-of-seeds gaps over the six instances, policy view):

algorithm	mean best gap	beats baseline on
---	---	---
alns (policy)	5.80%	6/6 instances
sa (tuned)	6.74%	4/6
ga_island (tuned)	7.28%	5/6
aco	7.56%	4/6
bnb_lds (small-instance mode)	7.77%	2/6
tabu	8.09%	5/6
baseline	8.70%	–



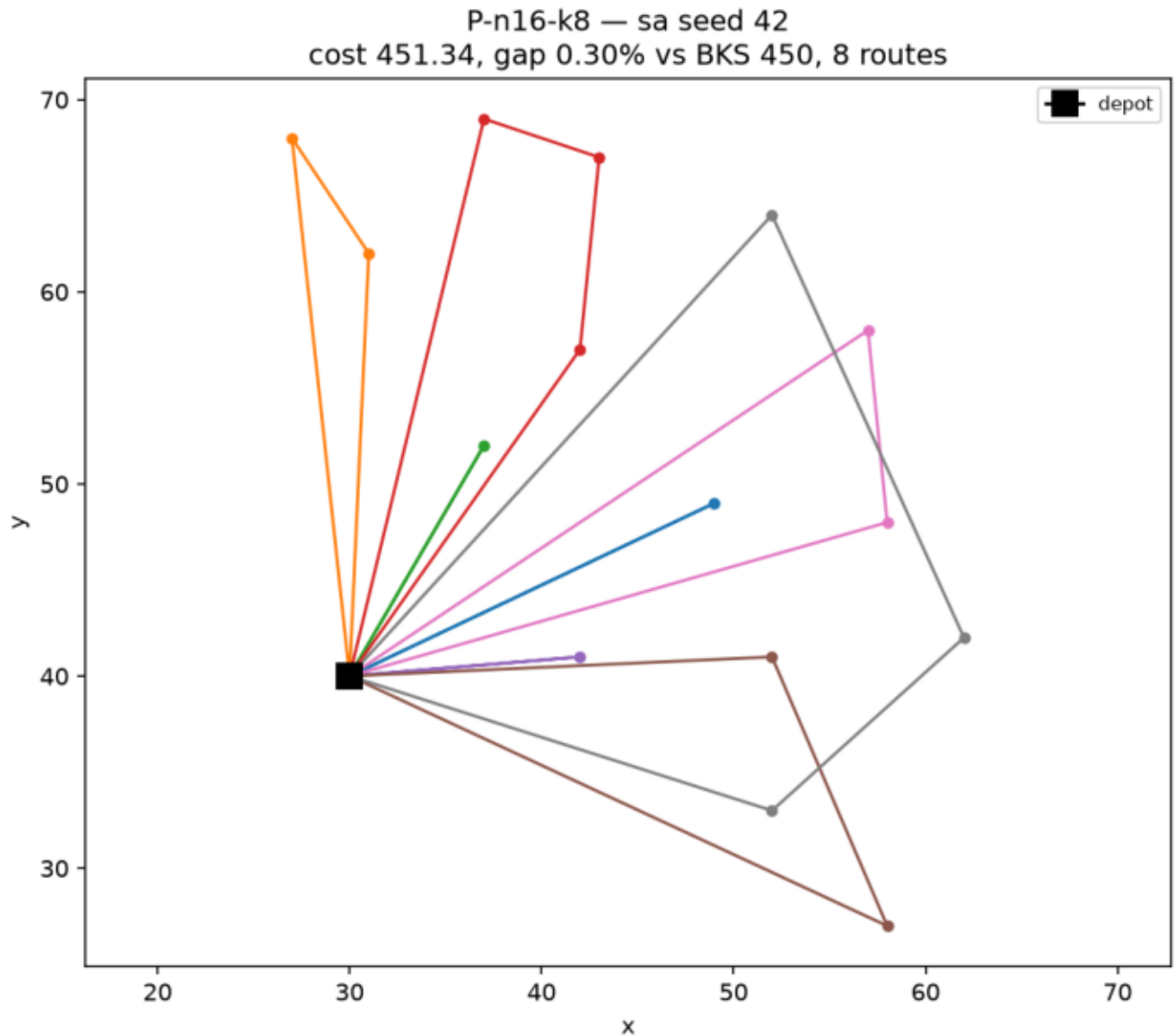
ALNS is now the only method that beats the multi-stage baseline on every single instance. The tuned SA is the biggest single mover (its mean best gap fell from 8.50% to 6.74% purely by spending its unused time budget on more iterations with a longer schedule), and the tuned GA improved to 7.28% – but neither dominates, and both still fall back to baseline level on A-n80-k10. B&B/LDS finally earned its place with a small-instance mode (deeper discrepancy budget for instances with at most 25 customers): it now ties the project-best results on P-n16-k8 (0.30%) and E-n22-k4 (0.07%) in about a second each – but on the four larger instances it still returns its starting incumbent, which is stated as-is.



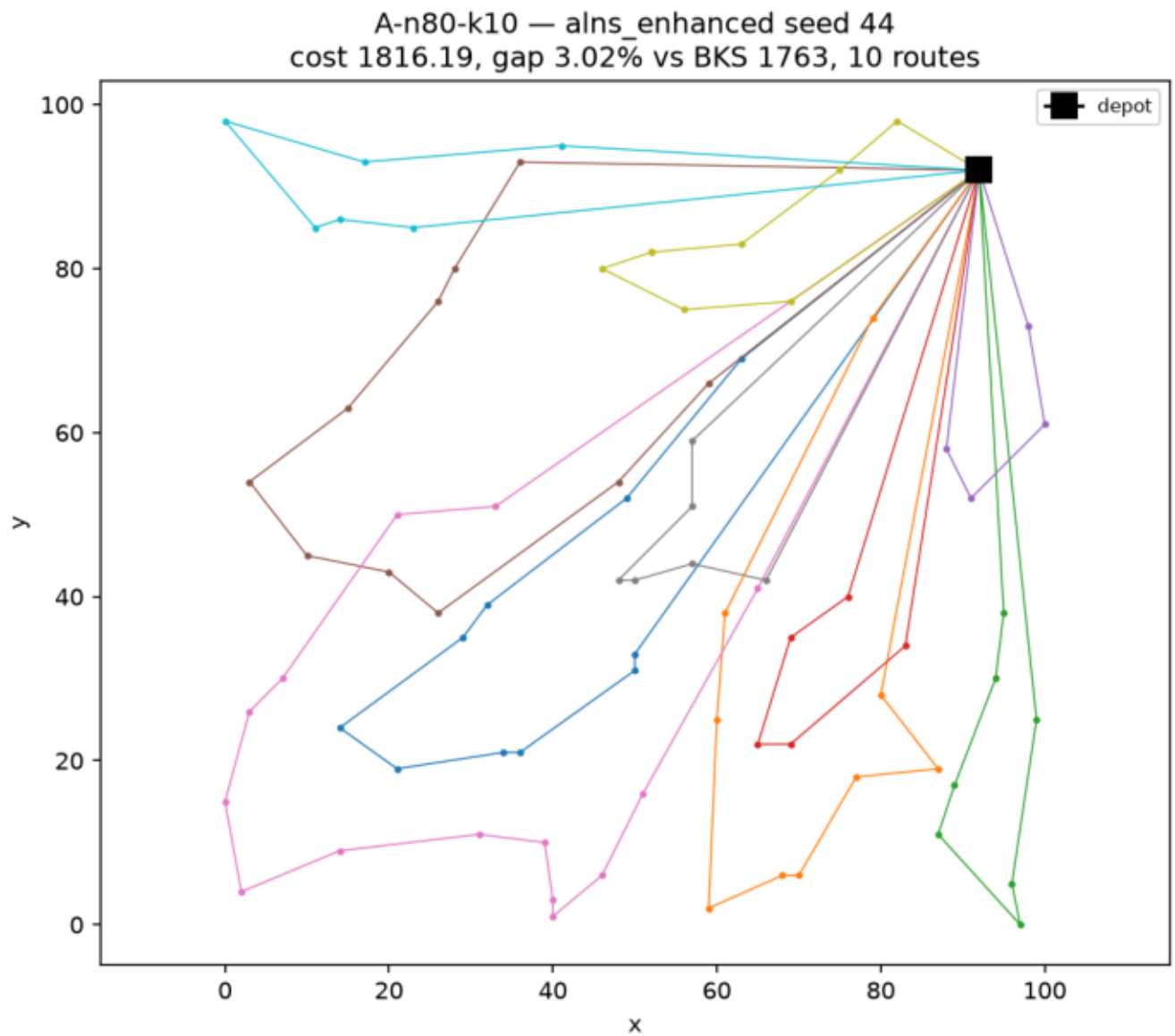
Mean runtime per run grows with instance size as expected (log scale). The values are averages over all seven algorithms, so the cheap methods (SA, baseline) pull the mean down; ACO alone accounts for most of the time on the two largest instances.

4.1 Route visualizations

Route plots are included to check feasibility visually on top of the validator checks: every tour must start and end at the black depot square, and the route count in the title must not exceed the fleet size.

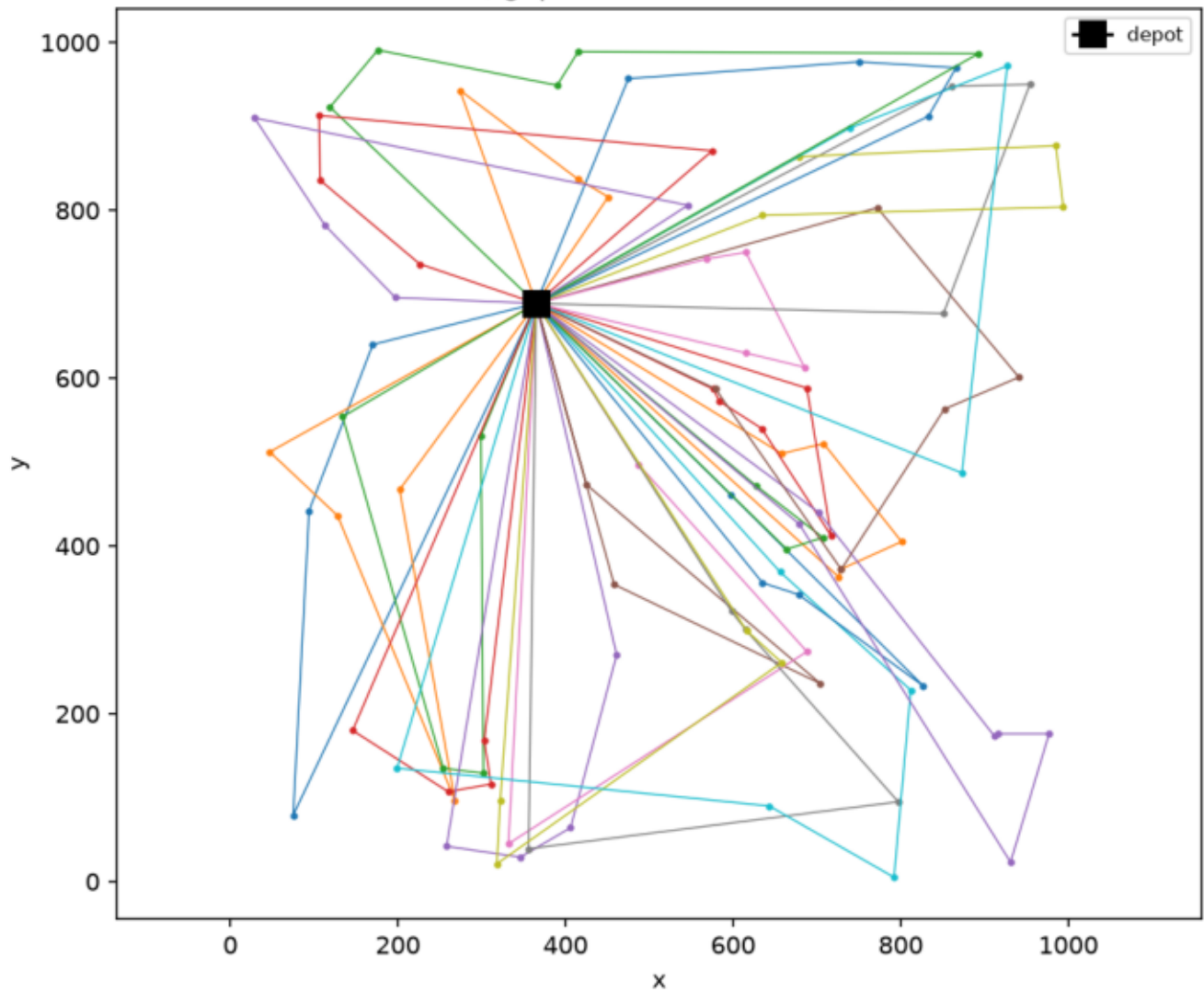


P-n16-k8 (tuned SA, seed 42, cost 451.35): eight short routes, each serving only one or two customers – with capacity 35 and demands up to 30-plus, most vehicles can take very little, which is why the instance needs all eight vehicles despite having only 15 customers.



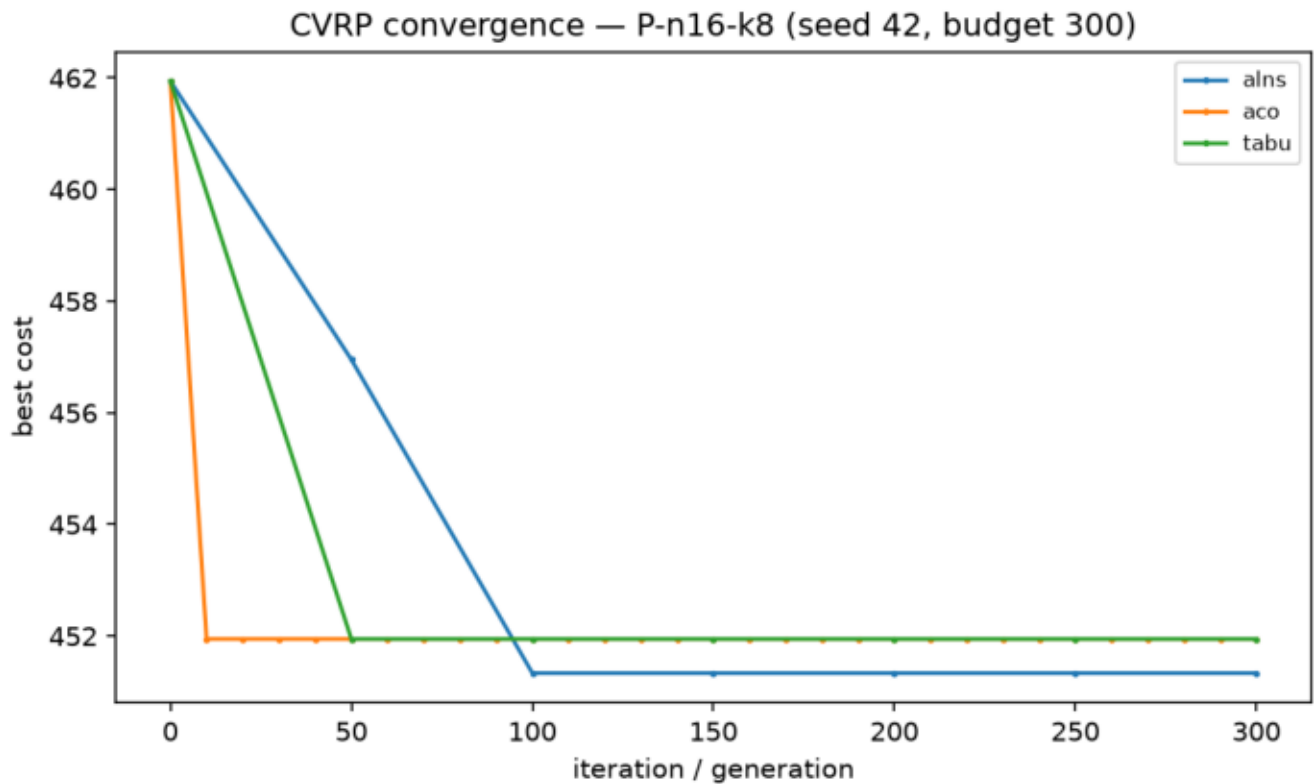
A-n80-k10 (enhanced ALNS, seed 44, cost 1816.28): ten routes fanning out of the depot in clean geographic sectors. Some crossings between neighboring routes remain – visual evidence of the remaining 3% gap that in-route 2-opt alone cannot remove.

X-n101-k25 — ga_island seed 43
cost 34266.65, gap 24.20% vs BKS 27591, 25 routes

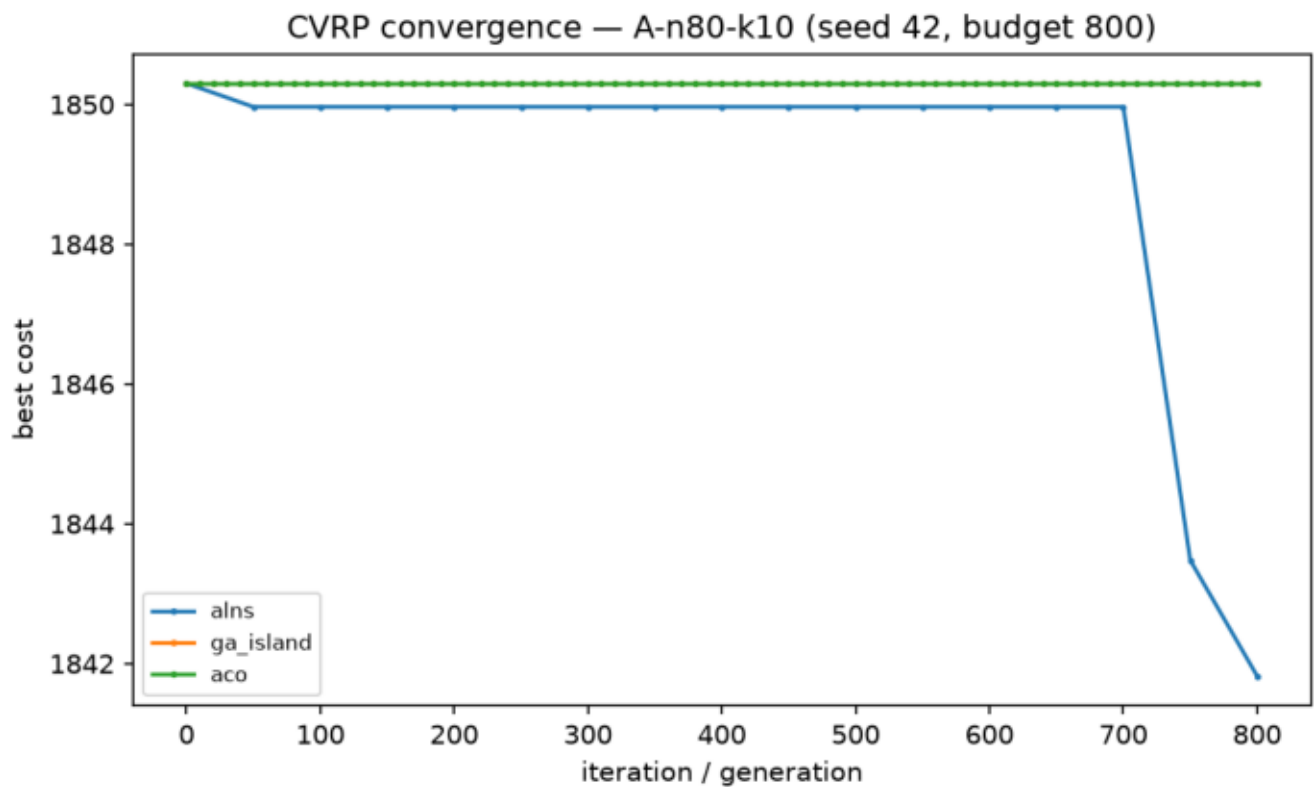


X-n101-k25 (tuned GA-Island, seed 43, cost 34269.89): the plot is dense because the instance is large (100 customers) and extremely tight (25 nearly-full routes), so it is harder to inspect visually – long criss-crossing legs are exactly what a load-driven packing looks like when geometry has to take second place to capacity.

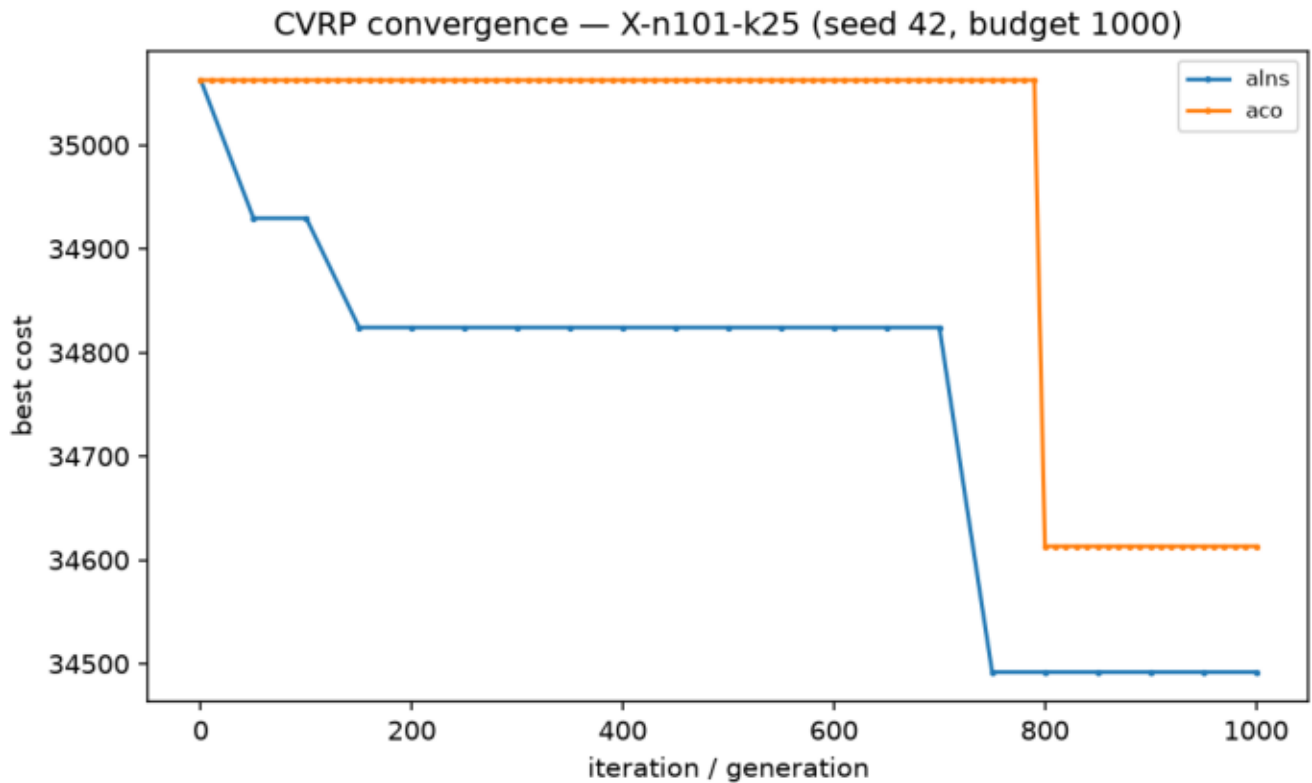
4.2 Convergence behavior



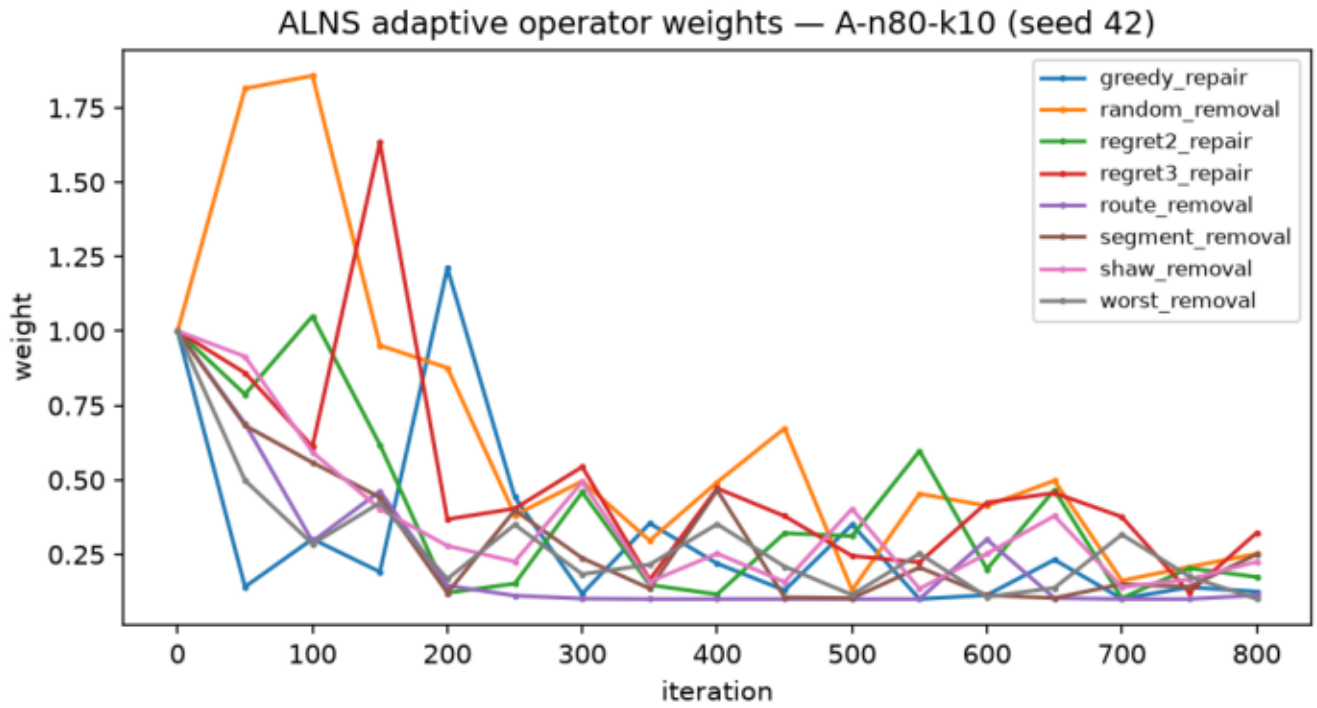
On the small instance every method starts from the repaired baseline (461.94) and improves quickly: ACO and Tabu reach 451.95 within the first 50 iterations, and the enhanced ALNS goes one step further to 451.34 for this seed. After that the curves are flat – the instance is essentially solved as far as these operators can take it.



On A-n80-k10 most of the quality still comes from the multi-stage baseline (1850.30), but the enhanced ALNS now visibly pulls away to 1841.82 for seed 42 (its best seed reaches 1816.28), while the tuned GA-Island and ACO stay on the baseline for this seed. The strong start compresses the visible improvement – the y-axis spans a handful of cost units.



On X-n101-k25 the curves confirm the packing story with one welcome change: the enhanced ALNS route-removal operator can now restructure whole routes, so ALNS improves the subset-sum start to 34491.83 for this seed (basic ALNS could not move at all in the 3-unit-slack packing), and ACO's constructive ants reach 34613.14. Neither gets anywhere near the BKS – improvement is still limited after feasible packing, as discussed above.

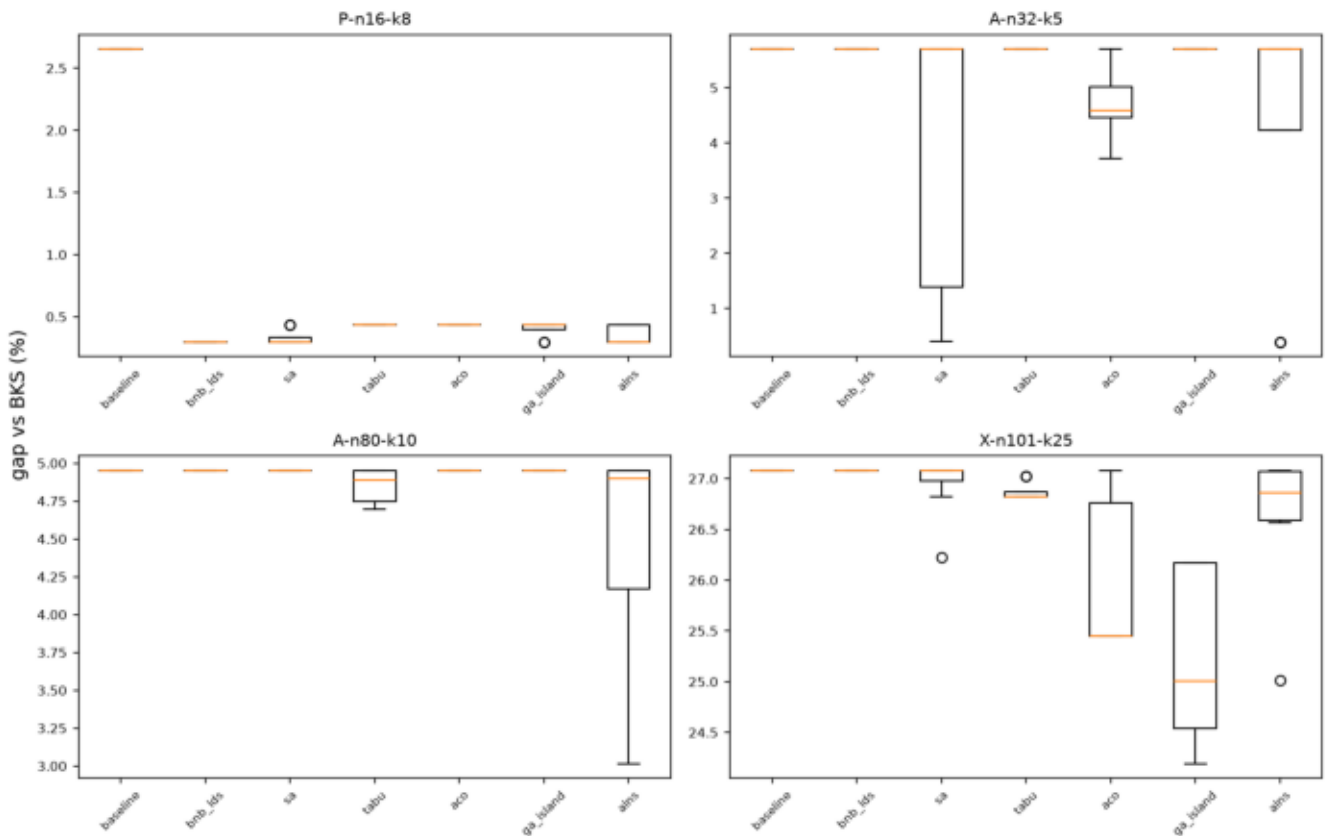


The ALNS adaptive layer is visibly working, now over the full enhanced pool (five destroy and three repair operators): weights move away from their initial 1.0 within the first ~50 iterations and then stabilize once improvements dry up. The drift toward the reject-score floor is expected behavior for a run that has already converged, not a malfunction.

4.3 Seed robustness (8 seeds)

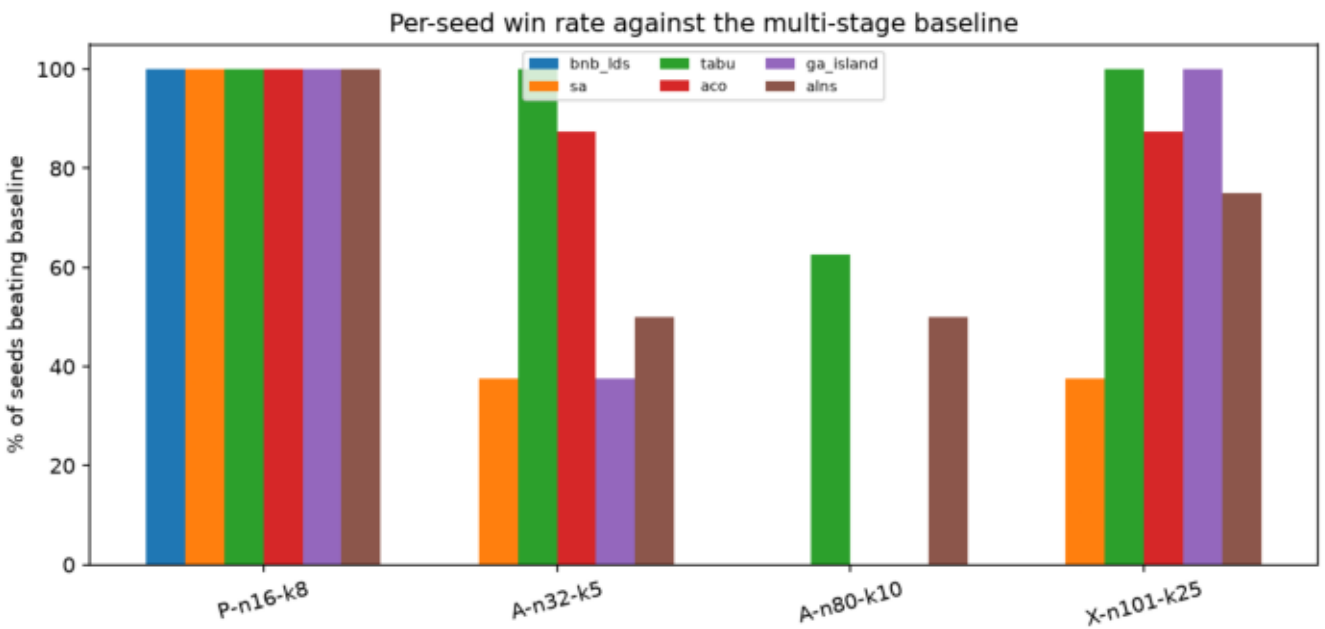
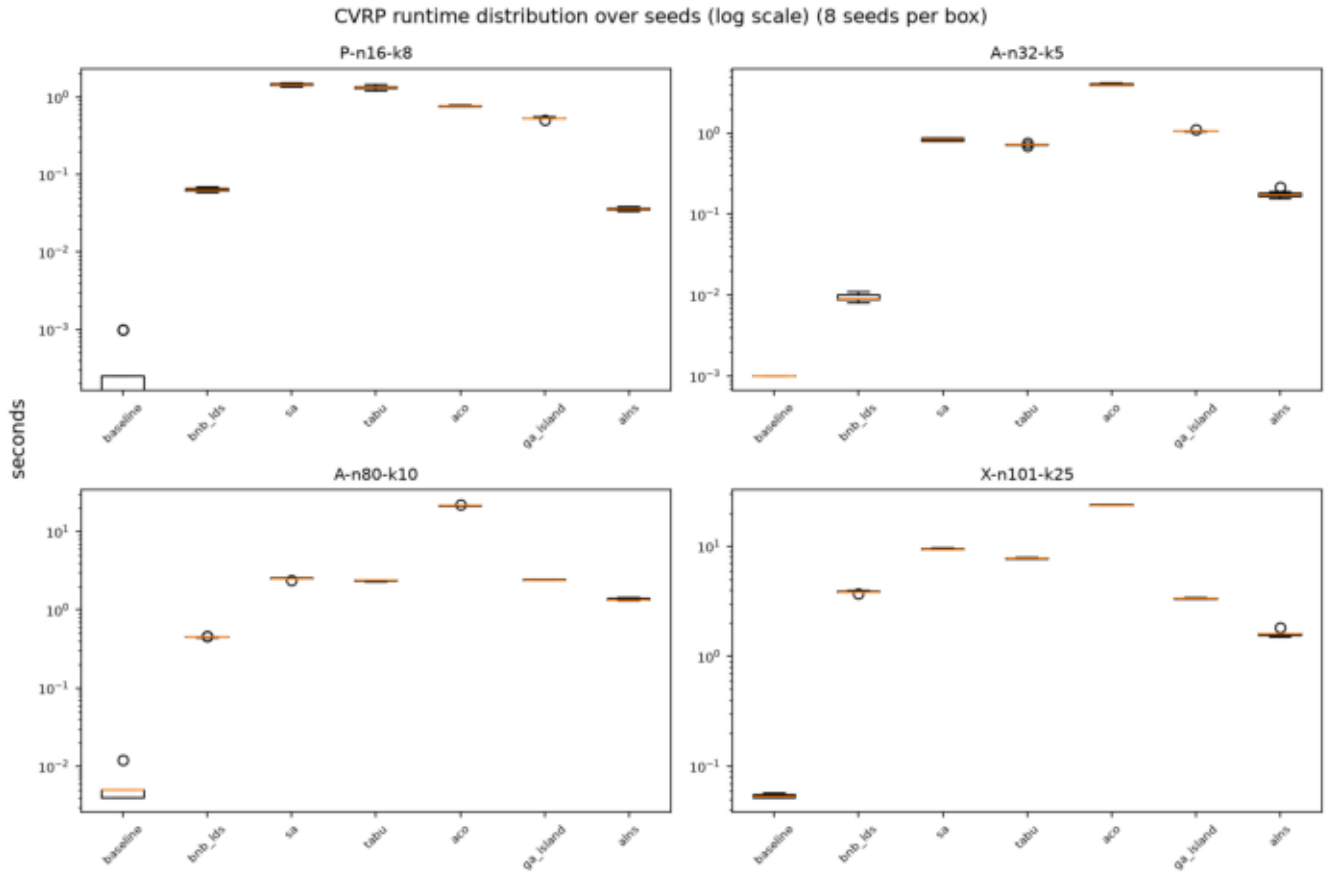
Best values can hide luck, so the final effective algorithm set was rerun with eight seeds (42–49) on four representative instances – 224 runs, all feasible. The boxplots show the full gap distribution, not just the best:

CVRP gap distribution over seeds (8 seeds per box)



Median-level reading, which is more honest than best-level:

- **P-n16-k8**: everything beats the baseline on every seed; B&B/LDS is the most robust of all (0.30% on all 8 seeds – it is deterministic), and SA/ALNS have medians at 0.30% with occasional 0.43% seeds.
- **A-n32-k5**: the headline 0.39–0.41% results of ALNS and SA are **seed-sensitive** – their medians sit at the baseline 5.70%, meaning most seeds do not escape it. ACO is quietly the most consistent improver there (median 4.58%).
- **A-n80-k10**: ALNS improves on most seeds (median 4.90%, best 3.02%); Tabu improves slightly and consistently (median 4.89%); everything else sits on the baseline.
- **X-n101-k25**: GA-Island is the most robust improver (median 25.01%, best 24.20%); ALNS helps on some seeds only (median 26.86%).



The win-rate chart summarizes the same story: on the small instance every method wins every seed; on the tighter instances only some methods win reliably. With 8 seeds these are descriptive statistics – no significance test is claimed – but they make one thing clear that the best-value tables cannot: ALNS's advantage is broad but not uniform, and some of the tuned wins (especially on A-n32-k5) depend on the seed.

Honest note on X-n101-k25: its total demand is 5147 while the total fleet capacity is $25 \times 206 = 5150$ – only 3 units of slack over 25 routes, so almost every route must be loaded completely full. Clarke-Wright needed 28

routes there, and a subset-sum packing repair (Section 5) was required just to reach feasibility. That packing ignores geometry, and with nearly zero capacity slack the usual local moves (relocate, cross-route swap) are almost always capacity-infeasible, so the metaheuristics could barely improve the start. The 25.45% gap is real and reported as such.

5. Multi-stage CVRP Heuristic

The baseline heuristic runs in five stages:

1. **Construction** with Clarke-Wright savings (merge route ends by descending saving under the capacity limit).
2. **2-opt** inside each route (reverse inner segments while improving).
3. **Relocate** between routes (move single customers while improving).
4. **Vehicle-count repair**: first empty surplus routes and reinsert their customers at cheapest feasible positions; if that fails, deterministic rebuilds (cheapest insertion, best-fit packing, sweep by polar angle); as a last resort a subset-sum packing that fills vehicles one at a time as full as possible over the integer demands – this is what makes X-n101-k25 feasible.
5. **Final validation** – a failed repair is returned as `feasible=False` with errors, never hidden.

Why the repair stage was necessary: Clarke-Wright merges routes only while the merge is profitable, so on instances with a tight fleet it can stop with more routes than vehicles exist. This actually happened twice in the final runs – P-n16-k8 (9 routes for 8 vehicles) and X-n101-k25 (28 routes for 25). Without the repair, every algorithm that starts from the baseline would inherit an infeasible incumbent, which is exactly what the first final run showed before stages 8-B2/9-B2. The subset-sum packing below is the piece that finally made X-n101-k25 feasible:

```
src/cvrp/local_search.py – subset-sum vehicle repair
```

```
349 | def build_routes_subset_sum_packing(instance, customer_order):
350 |     """Fill vehicles one at a time as full as possible, using a subset-sum
351 |     table over the integer demands.
352 |
353 |     Needed when total demand almost equals total capacity: X-n101-k25 has
354 |     5147 demand against 25 * 206 = 5150 capacity, so almost every route must
355 |     be loaded completely full, which greedy packing cannot find.
356 |     Returns customer groups or None.
357 |     """
358 |     capacity = int(instance.capacity)
359 |     if instance.capacity != capacity:
360 |         return None # table needs integer capacity
361 |     demands = {}
362 |     for customer in customer_order:
363 |         demand = instance.demands.get(customer, 0.0)
364 |         if demand != int(demand) or demand > capacity:
365 |             return None # table needs integer demands
366 |         demands[customer] = int(demand)
367 |
368 |     remaining = list(customer_order)
369 |     total_left = sum(demands[c] for c in remaining)
370 |     groups = []
371 |     for bins_left in range(instance.vehicle_count, 0, -1):
372 |         if not remaining:
373 |             break
374 |         # everything not in this bin must still fit into the other bins
375 |         lower = max(0, total_left - (bins_left - 1) * capacity)
376 |         if lower > capacity:
```

The idea is simple to state: when total demand almost equals total fleet capacity, almost every vehicle must leave completely full, so the repair fills vehicles one at a time with a subset of customers whose demands sum as close to the capacity as possible (a small dynamic-programming table over the integer demands), with a lower bound making sure the remaining customers still fit into the remaining vehicles.

Complexity: everything here is heuristic, not exact. One 2-opt pass over a route of length L costs $O(L^2)$ route evaluations and passes repeat until no improvement. The relocate pass scans all customer/position pairs, roughly $O(n^2)$ per pass. The repair adds packing work (the subset-sum table is $O(n \cdot \text{capacity})$ per vehicle) but guarantees the route count fits the fleet, which turned out to be essential on two of the six official instances.

6. CVRP Algorithms

All six start from the multi-stage baseline solution, share one random relocate/swap/2-opt neighborhood where applicable, and were run with the same per-instance budget and timeout for a fair comparison.

6.1 Simulated Annealing

Full solutions as states; one random neighbor per iteration; accept improvements always and worse candidates with probability $\exp(-\delta/T)$. The first run used an untuned schedule ($T=100$, cooling 0.995, iterations = plan budget) and mostly stayed near the baseline (mean best gap 8.50%). The tuning pass exploited how cheap one SA iteration is compared with the other methods: 50× more iterations, cooling 0.9995 and $T=500$ – still well inside the shared per-instance timeout. That alone dropped SA's mean best gap to 6.74% and made it the best method on P-n16-k8 (0.30%) and E-n22-k4 (0.07%). It still cannot beat the baseline on A-n80-k10 or M-n200-k17.

6.2 Tabu Search

Samples 40 candidates per iteration from the shared neighborhood, forbids recently visited solutions via a customer-sequence signature (FIFO tenure 30), with aspiration on a new global best. Tied for the best P-n16-k8 result (0.43%) and was solid on the small instances; mean gap 8.12%.

6.3 Ant Colony Optimization

Ants build routes customer by customer with probability proportional to $\text{pheromone} \cdot (1/\text{distance})$ under the capacity limit, light 2-opt per ant, evaporation plus deposits on the iteration and global best. ACO produced the best feasible X-n101-k25 result (25.45%) – its constructive nature sidesteps the frozen-local-moves problem there – at the price of the highest runtime (mean 32.8 s per run).

6.4 GA Island Model

Giant-tour chromosomes (customer permutations) split into routes by a capacity-aware scan; 0X crossover, swap/inversion mutation, tournament selection, elitism 1; two islands with ring migration every 20 generations. The tuned final settings raise the population from 12 to 30 and the mutation rate from 0.15 to 0.3 (tuning also exposed and fixed an initial-population bug that looped forever when the population was larger than the number of distinct permutations). Result: mean best gap 7.28%, beats the baseline on 5/6 instances, and holds the overall best X-n101-k25 result (24.20%) – but it still sits at baseline level on A-n80-k10.

6.5 ALNS

Destroy operators and repair operators chosen by adaptive weights, with simulated-annealing acceptance. The basic pool is random/worst removal plus greedy/regret-2 insertion; the enhanced pool adds Shaw-style related

removal, full-route removal, segment removal and regret-3 insertion. The final rerun executed both variants on every instance, and the reported "alns" result follows the rule declared before the rerun: enhanced everywhere except M-n200-k17, where validation had shown a +1.02% regression, so the basic variant is used there. With that policy ALNS is the strongest method overall: lowest mean best gap (5.80%), the only method beating the baseline on 6/6 instances, and the best result on A-n32-k5 (0.39%), A-n80-k10 (3.02%) and M-n200-k17 (5.99%) at a moderate runtime.

6.6 Branch-and-Bound / LDS

A time-limited search over customer insertion decisions (hardest customers first), where taking the k-th cheapest insertion costs k discrepancy, with a partial-cost bound against the incumbent. With the original shallow budget (discrepancy 3) it never beat its starting incumbent anywhere. The final version adds a **small-instance mode**: instances with at most 25 customers get discrepancy 15 and a 2M-node cap (still timeout-capped, still falling back to the incumbent). Measured effect: P-n16-k8 improves from 2.65% to 0.30% in ~0.03 s and E-n22-k4 from 3.34% to 0.07% in ~1.1 s – both tie the best results any metaheuristic found, and being a deterministic search, it hits them on every seed. The same depth was tested on A-n32-k5 (31 customers, depth 12, 8 s) without any improvement, so the threshold honestly stops at 25 customers; on the four larger instances B&B/LDS still returns its incumbent (overall mean gap 7.77%, beating the baseline on 2/6). It remains exact-inspired, not a full exact solver for these sizes.

7. Ackley Adaptations

SA and Tabu Search are natural continuous methods here (Gaussian steps; a tabu list of rounded points). GA-Island and ALNS operate directly on continuous vectors (blend crossover and Gaussian mutation; dimension-wise destroy and repair). ACO and B&B/LDS are honest discretizations (pheromone over bins per dimension; limited-discrepancy search over bin choices) – the canonical versions of these methods are combinatorial, and the report does not claim otherwise. Random search is only a sanity baseline. The results in Section 3 reflect exactly this: the methods whose adaptation points toward the origin (ALNS's toward-zero repair, B&B/LDS's zero-first bin order) reached it, the general-purpose ones (GA-Island, Tabu) got close, and untuned SA performed worst.

8. Part B – Rush Hour with GP and GEP

A* solves 6×6 Rush Hour boards with g = number of moves and a pluggable heuristic h. GP and GEP evolve h as expressions over three board features – distance of the red car to the exit, number of blocking cars, number of free exit cells – plus small constants, with protected operators (+, -, *, /, min, max, abs, neg, log). Fitness strongly rewards solved puzzles (10000 each) and penalizes expanded nodes, solution cost, timeouts and node-cap hits. Every evaluation runs under the safety caps from Section 2.3.

GP uses expression trees with subtree crossover and mutation. GEP is a separate framework: a linear genome with a head (functions or terminals) and a tail (terminals only), decoded as a Karva K-expression; its operators are point mutation and one-/two-point crossover on the flat gene string. The decoder is the heart of GEP – it reads the linear genome left to right and attaches children level by level, ignoring leftover symbols:

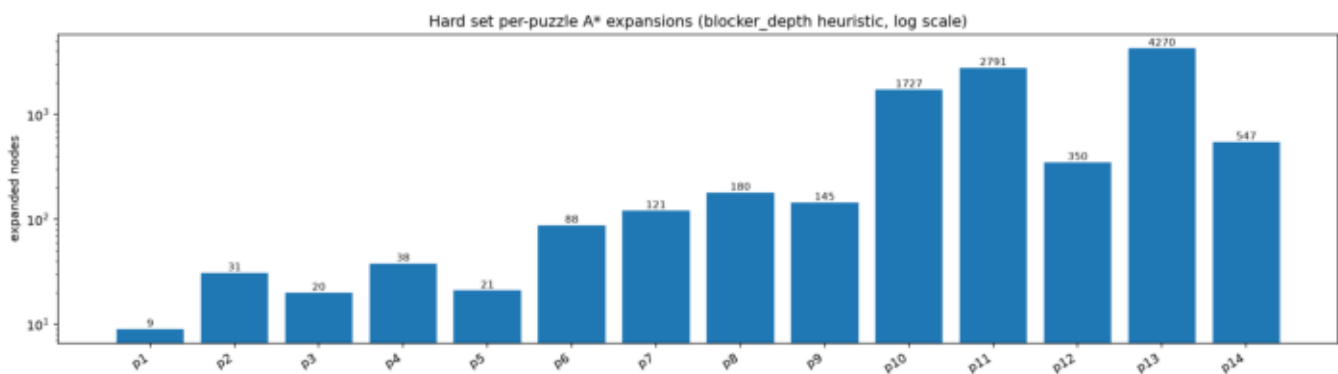
```
src/gep/decoder.py – Karva decoding
```

```
27 | def decode_genome_to_tree(genome: GEPGenome) -> GPTree:
28 |     symbols = genome.genes
29 |     root = _make_node(symbols[0])
30 |     open_nodes = deque()
31 |     if ARITY.get(symbols[0], 0) > 0:
32 |         open_nodes.append((root, ARITY[symbols[0]]))
33 |
34 |     index = 1
35 |     while open_nodes:
36 |         node, needed = open_nodes[0]
37 |         if len(node.children) == needed:
38 |             open_nodes.popleft()
39 |             continue
40 |         if index < len(symbols):
41 |             symbol = symbols[index]
42 |             index += 1
43 |         else:
44 |             symbol = "0.0" # genome ended early, pad with a safe terminal
45 |             child = _make_node(symbol)
46 |             node.children.append(child)
47 |             arity = ARITY.get(symbol, 0)
48 |             if arity > 0:
49 |                 open_nodes.append((child, arity))
50 |
51 |     return GPTree(root=root)
52 |
53 |
```

8.1 The hard benchmark (main Part B evaluation)

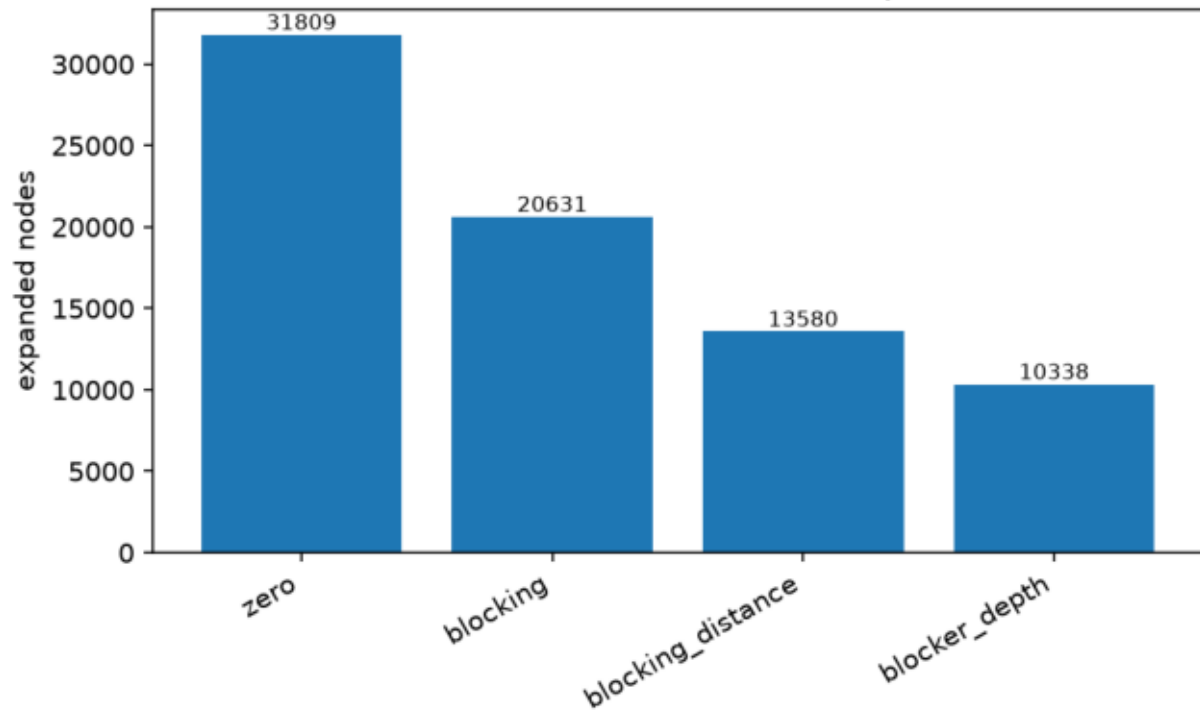
The first comparison used a 4-puzzle evaluation set, and everything – including plain manual heuristics – solved all of it with 4 expanded nodes, so GP and GEP tied perfectly at fitness 39956. That result is kept only as a historical smoke check; the main evaluation is a **hard Rush Hour benchmark** of 14 puzzles committed under

`examples/rushhour_hard_eval.txt`. Every puzzle was verified with the project parser and solved by the project A* before being committed, with optimal solutions of 4-17 moves and measured difficulty spanning 36 to 13,243 zero-heuristic expansions.



A ladder of manual heuristics anchors the comparison (identical caps for everything: 15,000 nodes and 2 s per puzzle, 20 s total per evaluation):

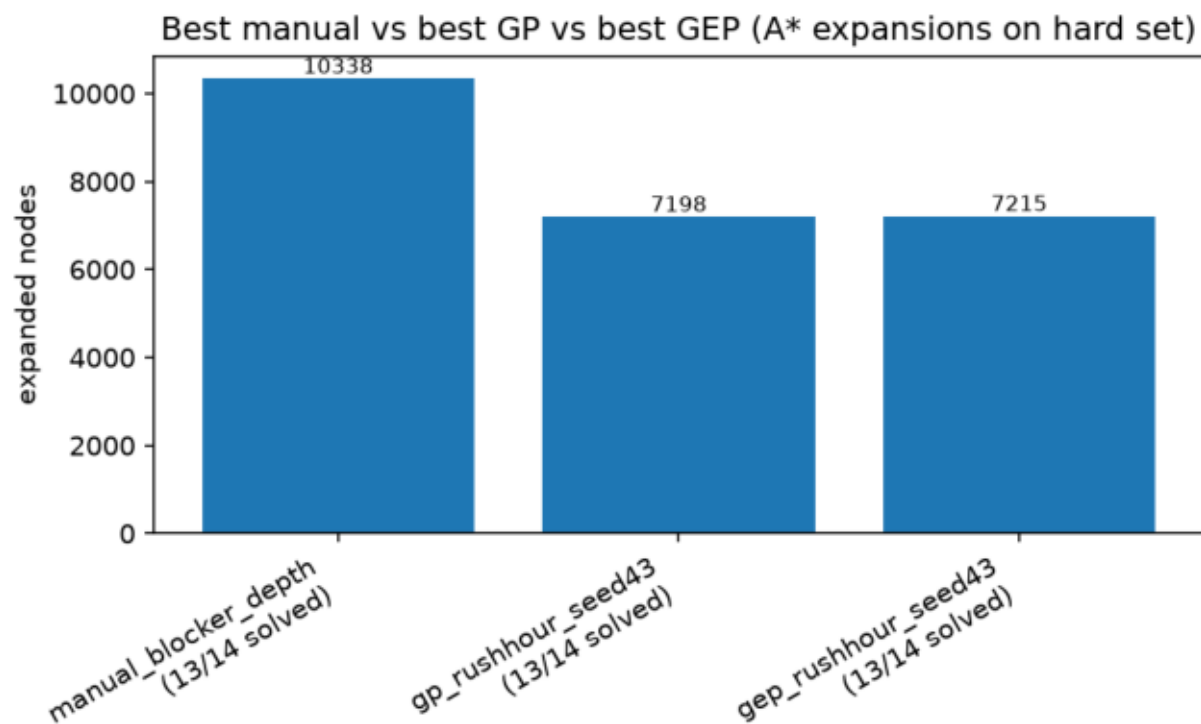
Rush Hour hard set: manual heuristic ladder (A* expansions, lower is bet



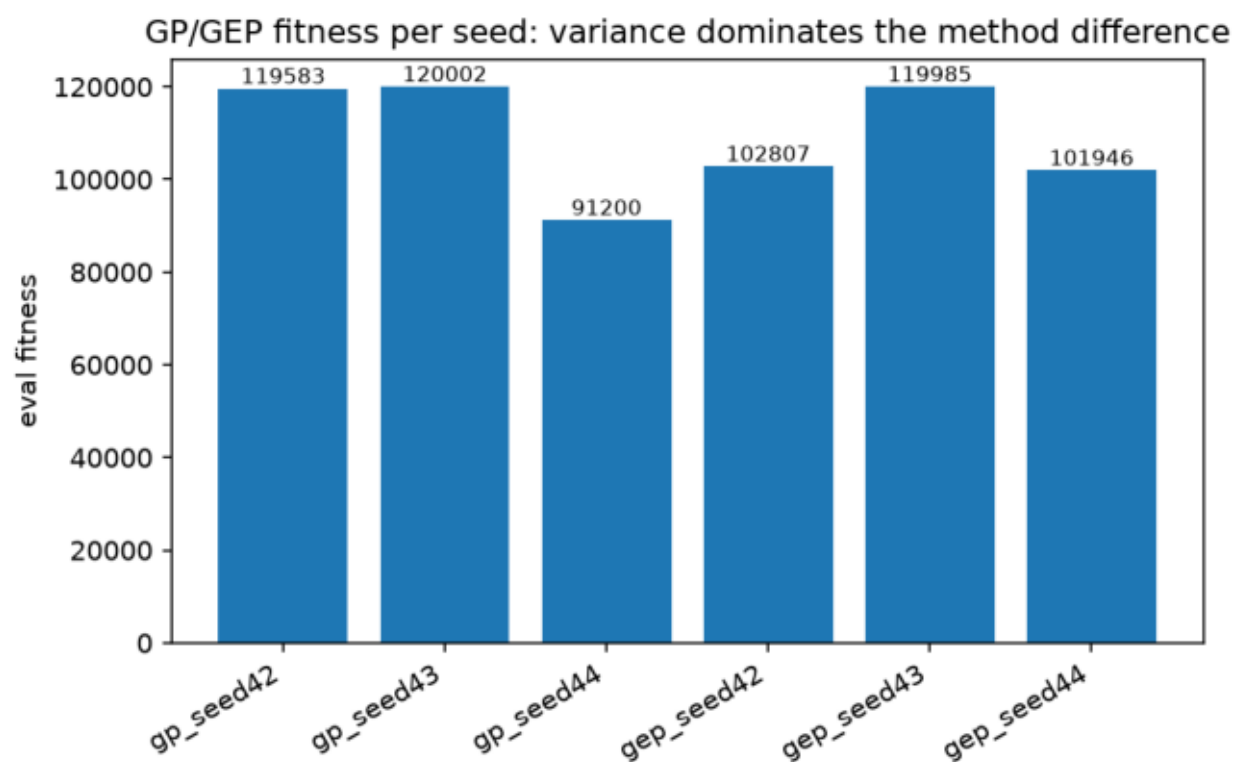
Each added ingredient pays off: zero solves only 9/14 within the caps (31,809 expansions), blocking gets 13/14 (20,631), blocking+distance needs 13,580, and blocker_depth – which also penalizes blockers that are themselves stuck – needs only 10,338. That monotone ladder is what makes the set a meaningful benchmark.

GP and GEP were then run with identical budgets (population 30, 20 generations, seeds 42-44; training on the 4 original puzzles plus the 3 easiest hard ones; evaluation on all 14):

run	eval fitness	solved	expanded	best expression
---	---	---	---	---
gp seed 42	119583	13/14	7617	$\min((\min(\text{blocking}, \text{blocking}) * (\text{distance} - 0)), \dots)$
gp seed 43	**120002**	13/14	7198	$(\text{abs}((\text{blocking} * (\text{blocking} * \text{blocking}))) / \text{abs}(0.5))$
gp seed 44	91200	11/14	12150	$((((\text{distance} * \text{distance}) / \text{free}) \dots * (\text{blocking} / 0.5))$
gep seed 42	102807	12/14	12533	$(\max(\max(1, 0), 5) - ((\text{free} - \text{distance}) * 2))$
gep seed 43	**119985**	13/14	7215	$\max((5 * \text{blocking}), ((1 + \text{distance}) * \text{blocking}))$
gep seed 44	101946	12/14	13384	$(\text{blocking} + (\text{distance} / (\text{abs}(\text{blocking}) - \text{free})))$
manual blocker_depth	116852	13/14	10338	(hand-written)



****The core finding:**** the best evolved heuristics from both frameworks beat the strongest manual heuristic – GP 120002 and GEP 119985 vs blocker_depth 116852 – solving the same 13/14 puzzles with about 30% fewer A* expansions (7,200 vs 10,338). Evolution genuinely found better guidance than the hand-written baselines here.



****GP vs GEP stays an honest tie at the top:**** 120002 vs 119985 is a 0.01% difference, while the per-seed spread within each method is ~30,000

fitness points (GP's worst seed drops to 91200, GEP's to 101946). Seed variance dominates the method difference, so no winner is declared. Expression diversity is 1.00 for both. Notably, no run solves 14/14 under the caps – the hardest puzzle needs more than the 15,000-node budget with any evolved or manual guide – which means the benchmark still has headroom and the results are not saturated like the old 4-puzzle set. (Numbers this close to the time caps can shift by a fraction of a percent between runs; the ordering has been stable.)

9. Results

The main tables and figures are in Sections 3 (Ackley), 4 (CVRP) and 8 (GP/GEP); the committed report figures live under ``report/figures/`` and small evidence snapshots under ``report/evidence/``. The raw per-run rows live under ``results/final_experiments/raw/`` (144 CVRP rows including both ALNS variants, 21 Ackley rows) plus the hard Rush Hour benchmark under ``results/final_experiments/rushhour_hard/``; the aggregated tables are under ``results/final_experiments/summary/``. The execution manifest (``final_execution_manifest.json``) records what ran, with which budgets, the tuned settings, and the pre-declared ALNS policy.

10. Analysis and Discussion

- ****CVRP pattern.**** Gaps grow with instance size: 0.30% or less on P-n16-k8 and E-n22-k4, 0.39% on A-n32-k5, ~3-6% on A-n80-k10 and M-n200-k17. After tuning, ALNS beats the baseline on all six instances (mean best gap 5.80%) and is the most consistent method overall.
- ****What tuning bought (and did not).**** P-n16-k8 0.43?0.30, A-n80-k10 4.03?3.02, X-n101-k25 25.45?24.20, with the other three unchanged and no regressions. The tuned SA improvement came purely from spending its unused time budget on more iterations – a fairness-preserving change, since all methods share the same timeout. The enhanced ALNS improvement needed real new operators (Shaw/route/segment removal, regret-3). Its M-n200-k17 regression was handled by the pre-declared hybrid policy, not by hiding rows.
- ****X-n101-k25.**** With 3 units of total capacity slack, feasibility is a bin-packing problem and local moves are nearly frozen: any relocate overfills a route. The enhanced ALNS route-removal operator and the tuned GA can now restructure a little (best gap 25.45?24.20), but the gap stays large. Truly improving it would need capacity-aware compound moves (e.g. ejection chains), which stayed out of scope.
- ****Ackley.**** Unchanged by this stage. The warm-up separated the methods, but part of that separation comes from how each was adapted (Section 7). Untuned SA losing to random search there remains a useful reminder that parameter choices matter as much as the algorithm name – the CVRP side proved the same point in reverse when tuned SA jumped from 8.50% to 6.74%.
- ****GP vs GEP.**** On the hard benchmark both frameworks beat the strongest manual heuristic by ~30% fewer expansions, and they remain effectively tied at the top (120002 vs 119985) with per-seed variance ~30,000 fitness points – far larger than the method difference. GEP's best expressions stay visibly more compact. Both frameworks are genuinely different representations, not renames.
- ****Runtime vs quality.**** ALNS delivers the best average CVRP quality at moderate runtime; ACO remains the most expensive; tuned SA uses its budget instead of leaving it idle. B&B/LDS with the small-instance mode ties the best results on the two smallest instances almost for free, but spends its budget without beating the incumbent everywhere else.
- ****Seed robustness.**** The 8-seed analysis (Section 4.3) shows which wins are stable: ALNS is a broad but not uniform improver, GA is the reliable one on X, B&B/LDS is perfectly stable where it works, and the A-n32-k5 headline numbers are seed-lucky (median = baseline).
- ****Limitations.**** One fixed budget/timeout profile per instance, three seeds for the main tables (eight for the robustness section), one tuning pass validated on the same six instances, and a 14-puzzle Rush Hour set – the comparisons hold for this setup only.

11. Complexity and Practical Considerations

- **Feasibility checking** is $O(n)$ per solution and is run on every candidate the metaheuristics accept, plus once on every final result – the cost is small compared to neighborhood evaluation and it caught real bugs (route-count violations) that pure cost comparison would hide.
- **Neighborhood costs.** The shared relocate/swap/2-opt neighborhood costs $O(n)$ per sampled neighbor (copy + cost); Tabu's 40 candidates per iteration make it $\sim 40 \times$ SA per iteration, which matches the observed runtimes.
- **Stochastic variance.** Three seeds per configuration; the summary CSVs report mean and standard deviation. The seeds are fixed in the plan, so every row can be regenerated exactly.
- **Timeouts and fairness.** All algorithms get the same timeout on the same instance, so slower-per-iteration methods are not silently given more work. Budgets differ per algorithm in meaning (iterations vs generations vs nodes), which is why the timeout is the binding fairness control on the large instances.
- **No optimality claims.** Everything here is heuristic or time-limited; the gaps against BKS quantify exactly how far the results are from the best known solutions.

12. Use of AI Tools

AI tools were used as coding and debugging assistants during the project. The final code and this report were reviewed and understood by the student. Any generated code was tested and adjusted through the project's test suite and staged workflow.

13. Reproducibility

- Python 3.12.3, dependencies via ``pip install -r requirements.txt`` (humpy, matplotlib, pandas, pytest).
- Place the six official CVRPLIB files under ``data/official_cvrp/`` and verify with ``python scripts/check_official_cvrp_data.py --strict``.
- Smoke check: ``python scripts/run_smoke_suite.py --output-dir results/smoke_suite``
- Print/validate the final plan:
``python scripts/print_final_experiment_plan.py --require-official-data``
- Full final tuned run (resumable):
``python scripts/run_final_experiments.py --tuned-cvrp configs/tuned_cvrp_settings.json --rushhour-hard configs/rushhour_hard_benchmark.json``
- Hard Rush Hour benchmark on its own:
``python scripts/run_gp_gep_hard_benchmark.py --puzzles examples/rushhour_hard_eval.txt --seeds 42 43 44``
- Old-vs-new comparison: ``python scripts/extract_final_results_v2.py``
- Report figures: ``python scripts/generate_report_figures.py``, plus ``python scripts/generate_route_visualizations.py`` and ``python scripts/generate_convergence_figures.py``, then ``python scripts/export_report_pdf.py`` for this PDF
- Report facts: ``python scripts/extract_report_numbers.py``
- Small evidence snapshots of the result files cited by this report are committed under ``report/evidence/`` (summaries, GP/GEP runs, execution manifest). The full generated outputs stay under ``results/final_experiments/`` locally and are not committed to Git, and the official ``vrp`` files are user-provided data that is also kept out of the repository.

The final runner is what makes the experiments reproducible in practice: it validates the plan and the official data before anything runs, executes each part through the same Python functions the tests use, writes one CSV row per run with its seed/budget/timeout, and skips already-finished raw files – when the vehicle-count repair changed, only X-n101-k25 had to be rerun while the other five instances were reused untouched:

```
src/experiments/final_execution.py - final suite
```

```
356 | def run_final_experiment_suite(plan_path="configs/final_experiment_plan.json",
357 |                               resume: bool = True, run_cvrp: bool = True,
358 |                               run_ackley: bool = True,
359 |                               run_rushhour: bool = True,
360 |                               tuned_cvrp_path=None,
361 |                               rushhour_hard_path=None) -> dict:
362 |     plan = load_final_plan(plan_path)
363 |     ok, errors = validate_final_plan(plan, require_official_data=True)
364 |     if not ok:
365 |         raise ValueError("final plan validation failed: " + " | ".join(errors))
366 |
367 |     tuned = None
368 |     if tuned_cvrp_path:
369 |         with open(tuned_cvrp_path) as f:
370 |             tuned = json.load(f)
371 |
372 |     output_dir = plan.get("output_dir", "results/final_experiments")
373 |     manifest = {
374 |         "plan_path": str(plan_path),
375 |         "resume": resume,
376 |         "run_cvrp": run_cvrp,
377 |         "run_ackley": run_ackley,
378 |         "run_rushhour": run_rushhour,
379 |         "tuned_cvrp_path": str(tuned_cvrp_path) if tuned_cvrp_path else "",
380 |         "tuned_cvrp_settings": tuned or {},
381 |         "alns_policy": (tuned or {}).get("alns_policy", {}),
382 |         "rushhour_hard_path": str(rushhour_hard_path) if rushhour_hard_path else "",
383 |         "cvrp_run_info": [],
384 |         "ackley_run_info": {},
385 |         "rushhour_run_info": {},
```

The submission audit output below is the real terminal output of the audit script on this repository – 28 checks covering files, row counts, feasibility, the report, and forbidden artifacts:

```

$ python scripts/audit_submission.py --check-results --check-pdf

PASS file exists: README.md
PASS file exists: requirements.txt
PASS file exists: report/assignment3_report.md
PASS file exists: configs/final_experiment_plan.json
PASS file exists: data/cvrp_bks.csv
PASS file exists: report/evidence/cvrp_all_summary.csv
PASS file exists: report/evidence/cvrp_all_policy_effective_summary.csv
PASS file exists: report/evidence/ackley_d10_summary.csv
PASS file exists: report/evidence/gp_gep_comparison_runs.csv
PASS file exists: report/evidence/rushhour_hard_manual_summary.csv
PASS file exists: report/evidence/rushhour_hard_gp_gep_summary.csv
PASS file exists: report/evidence/final_execution_manifest.json
PASS file exists: report/evidence/final_v2_summary.txt
PASS file exists: report/evidence/cvrp_seed_robustness_summary.csv
PASS file exists: report/assignment3_report.pdf
PASS report has no unresolved placeholders
PASS report has no overclaim phrases
PASS report has an AI tools section
PASS report covers the tuned rerun
PASS report covers the hard Rush Hour benchmark
PASS report shows before/after tuning figure
PASS report covers seed robustness
PASS result exists: results/final_experiments/raw/cvrp_all_instances.csv
PASS result exists: results/final_experiments/summary/cvrp_all_summary.csv
PASS result exists: results/final_experiments/raw/ackley_d10.csv
PASS result exists: results/final_experiments/summary/ackley_d10_summary.csv
PASS result exists: results/final_experiments/raw/gp_gep_comparison_runs.csv
PASS result exists: results/final_experiments/raw/gp_gep_comparison_summary.txt
PASS result exists: results/final_experiments/final_execution_manifest.json
PASS cvrp_all_instances has 144 rows (144 rows)
PASS all cvrp rows feasible
PASS no cvrp row has errors
PASS ackley_d10 has 21 rows (21 rows)
PASS gp_gep_comparison has 6 rows (6 rows)
PASS pdf size > 10 KB (1198931 bytes)
PASS forbidden absent: docs
PASS forbidden absent: AI_USAGE.md
PASS forbidden absent: report.md
PASS forbidden absent: report.pdf
PASS no ZIP files
PASS removed instance name absent
NOTE official .vrp files present: 6 (user-provided, not required to be tracked)

audit: PASS (0 failed check(s))

```

And the extracted report numbers, printed straight from the final CSVs (the same numbers used in the tables above):


```

$ python scripts/extract_report_numbers.py

cvrp rows: 144, feasible: 144, with errors: 0
  best P-n16-k8: cvrp_sa cost 451.3351 gap 0.2967%
  best E-n22-k4: cvrp_sa cost 375.2798 gap 0.0746%
  best A-n32-k5: cvrp_alns cost 787.0819 gap 0.3931%
  best A-n80-k10: cvrp_alns_enhanced cost 1816.1854 gap 3.0168%
  best X-n101-k25: cvrp_ga_island cost 34266.6469 gap 24.1950%
  best M-n200-k17: cvrp_alns cost 1351.4311 gap 5.9946%
mean gap baseline: 8.70%
mean gap cvrp_aco: 7.74%
mean gap cvrp_alns: 7.18%
mean gap cvrp_alns_enhanced: 7.31%
mean gap cvrp_bnb_lds: 7.77%
mean gap cvrp_ga_island: 7.38%
mean gap cvrp_sa: 7.43%
mean gap cvrp_tabu: 8.12%
ackley rows: 21, errors: 0
  best ackley: ackley_alns seed 42 value 0.000000
gp/gep rows: 6
  best: gp_rushhour seed 42 eval_fitness 39956.0 solved 4
  expression diversity over all runs: 1.00
  summary txt:
    gp runs: 3
    gep runs: 3
    gp expression diversity: 1.00
    gep expression diversity: 1.00
    gep genome diversity: 1.00
    best gp expression: min((min(blocking, blocking) * (distance - 0)), ((distance * blocking) / (blocking / distance)))
    best gep expression: (5 * blocking)
    best gep genome: * 5.0 blocking min blocking log 0.5 0.0 5.0 1.0 distance 2.0 distance
  These results depend on the small puzzle set and the selected random seeds.

```

14. Conclusion

The implementation covers everything the assignment asks for: the six required search algorithms on both the Ackley warm-up and the six official CVRP instances, an explicit multi-stage CVRP heuristic with an honest feasibility-repair story, and separate GP and GEP frameworks for evolving Rush Hour heuristics evaluated through A*. The final tuned rerun (144 CVRP rows, all feasible) improved the best gaps on three instances with no regressions: 0.07–0.39% on the three small instances, 3.02% on A-n80-k10, 5.99% on M-n200-k17, and 24.20% on the capacity-tight X-n101-k25 – still large and reported as a real limitation rather than smoothed over. ALNS with the pre-declared hybrid policy is the only method beating the multi-stage baseline on all six instances; B&B/LDS, after gaining a small-instance deep-search mode, ties the best known project results on the two smallest instances but still returns its incumbent on the four larger ones, which is stated as-is. On Ackley (unchanged), the adapted ALNS and B&B/LDS reached the optimum while untuned SA did not beat random search – a lesson the CVRP tuning then confirmed from the other direction. On the hard Rush Hour benchmark, both evolved frameworks beat the strongest manual heuristic by about 30% fewer A* expansions, while GP and GEP themselves remain honestly tied at the top with seed variance dominating. The main open improvements are capacity-aware compound moves for tight CVRP instances, a stronger bound to extend B&B/LDS beyond 25 customers, and pushing the hard Rush Hour set until the frameworks separate.